WISEConv: Worklist-driven Masked Convolution for Onboard High-Speed Robotic Perception

Anonymous Authors

***Abstract*—Onboard robotic perception relies on 2D convolutional networks that must meet hard latency constraints under the limited compute of embedded platforms. Many high-speed vision systems exploit spatial sparsity by producing an active mask and updating only the affected convolution outputs. However, existing approaches face a fundamental trade-off: tile skipping preserves the dense pipeline's throughput but computes many unneeded outputs, while gather-scatter computes only needed outputs but sacrifices throughput. Neither converts high sparsity into proportional latency reduction. We present WISEConv, a masked convolution operator that achieves both high useful-compute ratio and near-dense throughput by fusing position-level selectivity into the dense pipeline itself. WISEConv first scans the mask to build a spatially compact worklist of active positions, then drives the standard convolution datapath using only those positions. On Jetson Xavier NX and Orin, across three mask sources and three robotic perception tasks, WISEConv achieves near-linear latency scaling with the active ratio, outperforming state-of-the-art baselines while preserving task accuracy.**

***Index Terms*—Computer architecture for robotic and automation, software, deep learning methods, sparse convolution, embedded perception.**

I. Introduction

ONBOARD robots run convolutional perception in the control loop for navigation, manipulation, and closed-loop control [1]–[3]. On embedded platforms the available compute is far below server-class GPUs, yet perception latency is a hard real-time constraint because it sits directly on the control path. Dense convolution recomputes every spatial position, and on these platforms it often cannot reach the update frequency the loop demands. Many high-speed vision systems on UAVs, mobile robots, and embedded devices [4]–[7] therefore derive active masks from sources such as event-camera increments, temporal frame differences, or learned spatial gating, and use them to concentrate compute via *masked convolution*: the input stays a dense image-grid tensor, but only the outputs at positions marked active are updated.

The standard dense convolution pipeline partitions the output grid into *tiles* (contiguous spatial blocks) and for each tile reads the required input patch by coordinate, computes the convolution, and writes the result back. This coordinate-driven, tile-based execution achieves high throughput because tiles access spatially contiguous data with regular, predictable memory patterns. Existing masked-convolution systems modify or abandon this pipeline along two paths.

Tile skipping [4], [6], [8], [9] stays inside the dense pipeline: it tests each tile and skips it if every position within is inactive. Throughput is preserved for non-skipped tiles, but a single active position forces the entire tile to execute, so many computed outputs are unneeded. *Gather-scatter* [7], [10] abandons the pipeline entirely: it extracts active positions exactly, gathers their input neighborhoods, computes, and scatters results back. Every computed output is needed, but the extraction, irregular memory access, and scatter writeback collapse throughput.

We characterize this trade-off along two axes: *throughput* (operations per second, bounded by the dense pipeline) and *useful-compute ratio* (the fraction of executed operations that produce needed outputs). Tile skipping preserves throughput but sacrifices useful-compute ratio; gather-scatter achieves high useful-compute ratio but sacrifices throughput. Neither delivers both, so upstream sparsity does not translate into proportional latency reduction. In our evaluation on Jetson Xavier NX and Orin, existing methods [4], [6], [7], [10], [11] reduce the active-output ratio to roughly XX–YY%; yet tile-skipping methods achieve only AA–BB% useful-compute ratio, while gather-scatter methods reach only CC% of tile-skipping throughput. Convolution latency is reduced by only DD–EE%.

Our key insight is that position-level selectivity can be fused into the dense pipeline itself by replacing its coordinate source. Each tile already performs a coordinate-driven read-compute-write per position; substituting the tile's contiguous coordinate block with a pre-built set of active positions restricts computation to needed outputs while keeping the per-position datapath identical to the dense pipeline. This fusion achieves high useful-compute ratio without abandoning dense-level throughput.

Two problems must be solved to make this fusion practical. First, *coordinate discovery*: active positions must be identified before computation begins, yet scanning for them inside the compute kernel would stall the main datapath. Second, *locality preservation*: if active positions are fed in arbitrary order, memory access loses the spatial regularity that gives the dense pipeline its speed, and a global sort would itself erode the saved computation.

We present WISEConv (**W**ork**l****I**st-driven ma**S**k**E**d Convolution), a dense-grid masked convolution operator that resolves both overheads through a two-stage design. A lightweight construction stage scans the active mask and emits a spatially compact *worklist* of active output positions. The subsequent compute stage drives the dense pipeline's coordinate-driven execution with the worklist as its coordinate source, computing only the active outputs. Both overheads are addressed by the construction stage's tiled scan: it operates entirely in 2D mask space (no feature data touched), so coordinate discovery costs only XX% of the compute-stage time; and it compacts active positions within each tile into contiguous worklist

Manuscript received Month DD, YYYY.

segments. Consecutive entries within a segment are spatially nearby and often share overlapping receptive fields, partially recovering the locality of dense traversal without a global sort. With construction overhead minimized and only active outputs computed, WISEConv achieves near-linear latency scaling: an active-output ratio of $x\%$ yields convolution latency close to $x\%$ of the dense baseline.

This paper makes the following contributions:

- We identify the useful-compute ratio vs. throughput trade-off as the fundamental bottleneck preventing masked convolution from delivering proportional speedup on embedded platforms [4], [6]–[10].
- We present WISEConv, a worklist-driven masked convolution operator that fuses position-level selectivity into the dense pipeline's coordinate-driven execution, achieving both high useful-compute ratio and near-dense throughput.
- On Jetson Xavier NX and Orin, WISEConv achieves near-linear latency scaling with active-output ratio across three mask sources and three robotic perception tasks, outperforming dense cuDNN, tile skipping, and gather-scatter while preserving task accuracy.

II. Background and Motivation

A. 2D Dense Convolution on Embedded Platforms

Convolutional neural networks (CNNs) are the computational backbone of onboard robotic visual perception. CNN-based depth estimation and optical flow [1], [2], [12] feed obstacle avoidance and path planning; object detection [13] and semantic segmentation [14] provide scene understanding for navigation; and pose estimation [3], [15] enables closed-loop manipulation. While vision transformers [16] have achieved strong accuracy on these tasks, their quadratic attention cost makes them impractical under the tight latency and power budgets of embedded platforms; CNN-based models remain the dominant deployed choice for real-time onboard visual inference. Across these models, convolutional layers operating on image-grid feature maps dominate the inference workload and sit directly on the control-critical path.

On embedded platforms (Jetson Xavier NX, Orin, edge TPUs), power and thermal budgets limit available compute to a fraction of server-class GPUs, yet the closed control loop imposes a hard latency ceiling: perception must complete within the sensor frame interval (typically tens of milliseconds) to feed downstream planning and control in time.

Consider a CNN with L convolution layers. We describe the operation of a single layer l; all per-layer quantities (H_{out}, W_{out}, C_{in}, C_{out}, k, M, etc.) are implicitly indexed by l but we omit the superscript for clarity unless comparing across layers.

A 2D convolution layer transforms an input feature map $\mathbf{x} \in \mathbb{R}^{H_{in}\times W_{in}\times C_{in}}$ into an output $\mathbf{y} \in \mathbb{R}^{H_{out}\times W_{out}\times C_{out}}$ using a weight tensor $\mathbf{w} \in \mathbb{R}^{C_{out}\times C_{in}\times k\times k}$, where k is the kernel size and $k\times k$ is the receptive field size. The computation is defined per spatial position $p = (h, w)$ in the output: for each p, let $\mathcal{N}(p)$ denote the k^2 input positions in the receptive field of p. The layer computes:

$$\mathbf{y}[p] = \sum_{i=1}^{k^2} \mathbf{w}_i \, \mathbf{x}[\mathcal{N}_i(p)], \tag{1}$$

where $\mathcal{N}_i(p)$ is the i-th position in the receptive field and $\mathbf{w}_i \in \mathbb{R}^{C_{out}\times C_{in}}$ is the corresponding weight slice. The output at each position is thus an accumulation of k^2 weighted contributions from the input.

To formalize the GPU execution model, we define the *output index grid* $G_{out} \in \mathbb{Z}^{H_{out}\times W_{out}}$ with entries $G_{out}[h, w] = h \cdot W_{out} + w$, assigning each output spatial position a unique row-major linear index. The GPU partitions this grid into $N = N_h \cdot N_w$ disjoint tiles of size $t_h \times t_w$, where $N_h = \lceil H_{out}/t_h \rceil$ and $N_w = \lceil W_{out}/t_w \rceil$. Tiles are enumerated in row-major order as $T_1, \ldots, T_N$. The n-th tile occupies row $i_n = \lfloor (n-1)/N_w \rfloor$ and column $j_n = (n-1) \bmod N_w$ of the tile grid, and is defined as the row-major flattening of its slice:

$$T_n = \mathrm{vec}\big(G_{out}[i_n t_h{:}(i_n+1)t_h,\ j_n t_w{:}(j_n+1)t_w]^\top\big), \tag{2}$$

yielding a vector of $t_h \cdot t_w$ linear indices. Tiles are processed in parallel; within each tile, consecutive entries correspond to spatially adjacent positions whose receptive fields overlap and lie in contiguous memory. This regularity enables cuDNN-class [17] throughput. We denote by $\mathcal{C}$ the set of positions actually computed by a given execution strategy. Dense convolution computes all $H_{out} \times W_{out}$ positions unconditionally, which on embedded platforms often exceeds the latency budget.

B. Masked Convolution for Onboard Perception

1) Mask Sources: Many onboard vision systems exploit the fact that only a subset of the input changes between inference calls or is relevant to the current decision. These systems identify an input active mask $M_{in} \in \{0, 1\}^{H_{in}\times W_{in}}$ marking positions where the input has changed or is task-relevant. An output position p must be recomputed whenever its receptive field contains an active input position, giving the output active mask $M \in \{0, 1\}^{H_{out}\times W_{out}}$:

$$M[p] = \begin{cases} 1 & \text{if } \exists\, r \in \mathcal{N}(p) \text{ s.t. } M_{in}[r] = 1, \\ 0 & \text{otherwise.} \end{cases} \tag{3}$$

In a multi-layer network, the output mask of layer l becomes the input mask of layer $l{+}1$, propagating sparsity through the model. Three families of mask sources produce the initial M_{in}:

a) Event-camera increments.: Event cameras [18] are bio-inspired sensors in which each pixel independently fires when its brightness change exceeds a threshold, generating data only where intensity varies. They offer microsecond temporal resolution, high dynamic range, and negligible motion blur, making them widely used on UAVs and high-speed platforms. Events accumulated over a temporal window are differenced to reveal which positions received new activity, producing a naturally sparse mask [4], [5].

b) Temporal frame differences.: Standard frame-based cameras on mobile robots and autonomous vehicles capture video at a fixed rate. Between consecutive frames most of the scene remains static, especially under moderate ego-motion. Temporal differencing methods [6], [11] compare consecutive frames (or motion-compensated histories) and identify positions where pixel differences exceed a threshold; only outputs affected by these positions need recomputation.

c) Learned spatial gating.: Rather than relying on temporal change, dynamic networks [7] train a lightweight gating module that examines the current input and predicts, per spatial region, whether computation is needed for the downstream task. Regions classified as low-value are masked out, adapting the compute budget to task-relevant content per frame.

2) Execution Paradigms: Given a mask M, the system must compute $\mathbf{y}[p]$ only for positions where $M[p] = 1$. Existing methods follow one of two paradigms:

a) Tile skipping.: A tile T_n is executed if and only if it contains at least one active position; otherwise it is skipped entirely. The computed set is $\mathcal{C} = \bigcup_{\{n:\, \exists\, g \in T_n,\, M[g]=1\}} T_n$. Representative systems include SBNet [8], Skip-Convolutions [9], Ev-Conv [4], and DeltaCNN [6].

b) Gather-scatter.: Active positions are extracted exactly: the computed set $\mathcal{C} = \{g : M[g] = 1\}$ contains only active positions. DGNet [7] and SAI [10] follow this paradigm.

C. The Gap: Sparsity Does Not Become Speedup

For a network with L convolution layers, let $\|M^l\|_0$ denote the number of active positions (nonzeros) in layer l's output mask. We define layer l's *active ratio* $\rho^l = \|M^l\|_0/(H^l_{out}W^l_{out})$ and the network-level active ratio $\hat{\rho} = \frac{1}{L}\sum_{l=1}^{L}\rho^l$, the average fraction of output positions that require computation across all layers. We characterize masked-convolution efficiency along two axes: *throughput*, the rate of convolution operations per second (bounded by the dense pipeline), and *useful-compute ratio* $\eta = \|M\|_0/|\mathcal{C}|$, the fraction of executed operations that produce needed outputs. Tile skipping preserves throughput but sacrifices η; gather-scatter achieves high η but sacrifices throughput. Neither delivers both, so upstream sparsity does not translate into proportional latency reduction. Figure 1 quantifies this on Jetson Orin running XX on YY dataset: with $\hat{\rho}$ in the range XX–YY%, tile-skipping methods achieve only AA–BB% η and gather-scatter methods reach only CC% of tile-skipping throughput, reducing convolution latency by only DD–EE% rather than the ideal $\hat{\rho}$-linear scaling. This gap motivates WISEConv.

D. 3D Sparse Convolution: A Different Problem

3D sparse convolution (SpConv v2 [19], TorchSparse++ [20]) targets point-cloud workloads where input data is inherently irregular and the core challenge is imposing computational structure on scattered voxels. WISEConv faces a fundamentally different problem: the input resides on a regular dense grid with structured, stride-based access; the challenge is how to skip inactive outputs without leaving the dense pipeline's already-efficient execution. Applying 3D sparse convolution directly to this 2D scenario performs poorly because it requires building coordinate hash tables and neighbor maps with mandatory GPU synchronization, and the per-frame mask changes prevent amortizing this cost across layers. On Jetson Orin, SpConv v2 applied to a 2D masked convolution with $\rho^l = $ XX% runs XX$\times$ slower than the dense baseline due to construction overhead alone (Section V).

[TODO] Grouped bar chart on Jetson Orin. Three groups: Dense, Tile skipping, Gather-scatter. Each group has three bars for three metrics. Left y-axis: latency (ms). Right y-axis 1: useful-compute ratio η (%). Right y-axis 2: throughput (GFLOPS). Dashed horizontal line at $\hat{\rho}$ on the η axis, marking the active ratio (coincides with dense η bar top). Shows: tile skipping has low η; gather-scatter has low throughput; neither achieves latency proportional to $\hat{\rho}$.

Fig. 1. Efficiency profile of existing masked-convolution paradigms on Jetson Orin. Dense serves as the throughput and latency reference. Tile skipping preserves throughput but suffers low η; gather-scatter achieves high η but sacrifices throughput. Neither translates sparsity into proportional latency reduction.

III. WISEConv Design

A. Overview

WISEConv resolves the useful-compute ratio vs. throughput tension (Section II-C) through a two-stage design (Figure 2). Each WISEConv layer takes as input the dense feature map $\mathbf{x}$ and the input active mask M_{in} (Section II-B), and produces the output feature map $\mathbf{y}$ (valid at active positions) together with the output mask M for the subsequent layer:

1) **Construction stage.** A lightweight kernel scans the input active mask M_{in} and evaluates Eq. (3) for each output position, emitting a spatially compact *worklist* $\mathbf{q} = (q_1, \ldots, q_m)$ of $m = \|M\|_0$ active output indices drawn from G_{out}. The worklist exposes tile-local spatial locality (Section III-B).
2) **Compute stage.** A convolution kernel consumes $\mathbf{q}$ as its coordinate source. For each q_j, the kernel decodes the spatial coordinate, reads the receptive-field input $\mathbf{x}[\mathcal{N}(q_j)]$, and accumulates Eq. (1) identically to the dense pipeline (Section III-C).

Both stages execute once per layer, since the output mask M differs at each layer due to mask propagation (Eq. (3)). The computed set $\mathcal{C} = \{q_1, \ldots, q_m\}$ contains only active positions (high useful-compute ratio), while the per-position datapath remains that of the dense pipeline (near-dense throughput).

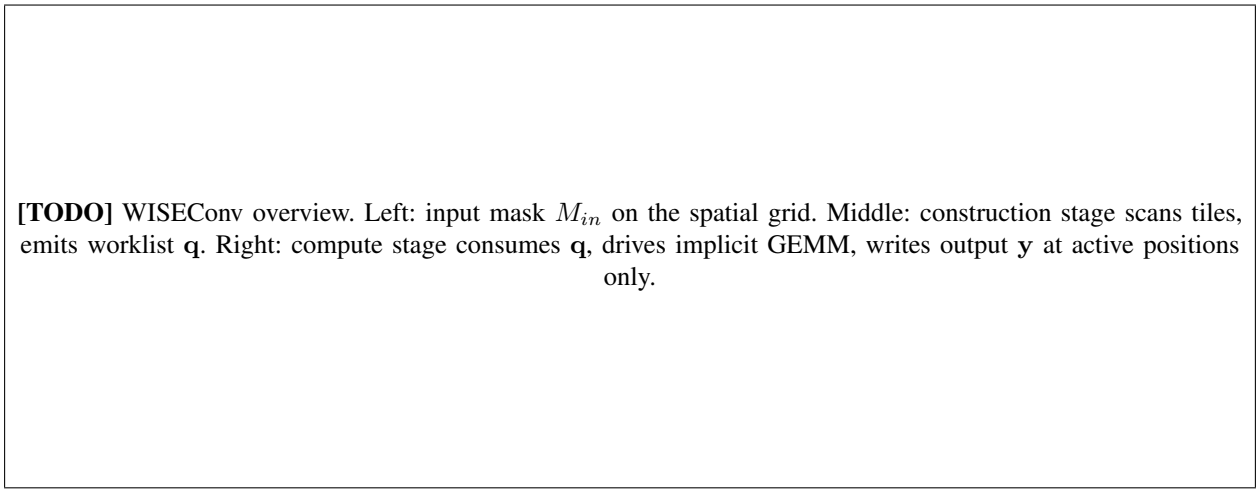

Fig. 2. WISEConv two-stage pipeline. The construction stage scans the mask tile by tile and emits a worklist of active positions. The compute stage drives the dense convolution datapath using only the worklist positions.

The construction stage operates entirely in mask space and its cost is independent of channel dimensions, making its overhead negligible relative to the compute stage.

B. Worklist Construction

The construction stage discovers active coordinates and arranges them to preserve tile-local spatial locality, with negligible overhead relative to the compute stage it feeds. It produces the worklist $\mathbf{q}$ from the input mask M_{in} by performing a per-tile gather followed by concatenation over the tile partition $T_1, \ldots, T_N$ defined in Eq. (2). For each output position, the stage evaluates Eq. (3) to determine the output mask M (requiring k^2 mask comparisons per position) and then gathers active indices within each tile in parallel.

For each tile T_n, the stage selects positions where M is active, yielding a support set:

$$\mathcal{I}_n = \big\{\, k \in \{1, \ldots, t_h t_w\} : M[T_n[k]] = 1 \,\big\}. \tag{4}$$

It then gathers the corresponding output indices from T_n in row-major order:

$$\mathbf{q}_n = (T_n[k])_{k \in \mathcal{I}_n}, \tag{5}$$

with length $|\mathbf{q}_n| = |\mathcal{I}_n|$.

Once all tiles complete, the stage concatenates their subsequences into the final worklist. Let $P = (P_1, \ldots, P_N)$ be a permutation of $(1, \ldots, N)$ determined by nondeterministic parallel block scheduling. The stage writes:

$$\mathbf{q} = \big[\, \mathbf{q}_{P_1};\ \mathbf{q}_{P_2};\ \cdots;\ \mathbf{q}_{P_N} \,\big], \tag{6}$$

a contiguous array of $|\mathbf{q}| = \|M\|_0$ active output indices. Each segment $\mathbf{q}_{P_n}$ preserves its internal row-major order; only the inter-segment ordering depends on the scheduling permutation P. The above is summarized in Algorithm 1: lines 3–4 compute M, lines 5–6 perform the per-tile gather, and line 7 concatenates segments into $\mathbf{q}$. Computing M and building $\mathbf{q}$ are thus fused into a single kernel with no separate mask-propagation pass.

Algorithm 1 Worklist Construction

Require: Tiles $T_1, \ldots, T_N$, input mask M_{in}
Ensure: Worklist $\mathbf{q}$, output mask M
1: **for** each tile T_n **in parallel do**
2: **for** each $k \in \{1, \ldots, t_h t_w\}$ **do**
3: $p \leftarrow$ spatial position of $T_n[k]$
4: $M[p] \leftarrow$ Eq. (3) ▷ test $\mathcal{N}(p)$ against M_{in}
5: **end for**
6: $\mathcal{I}_n \leftarrow \{k : M[T_n[k]] = 1\}$
7: $\mathbf{q}_n \leftarrow (T_n[k])_{k \in \mathcal{I}_n}$
8: **end for**
9: $\mathbf{q} \leftarrow (\mathbf{q}_{P_1}, \ldots, \mathbf{q}_{P_N})$ ▷ P from block scheduling

The construction cost is dominated by mask evaluation. The construction stage evaluates Eq. (3) for all $H_{out}W_{out}$ output positions, performing k^2 mask comparisons per position. Its total work per layer is $O(H_{out}W_{out} \cdot k^2)$, independent of C_{in} and C_{out}.

To estimate the overhead relative to computation, we model per-layer latency as $\hat{t} = \hat{t}_{\text{construct}} + \hat{t}_{\text{compute}}$, assuming each is proportional to its operation count. Construction performs $H_{out}W_{out} \cdot k^2$ operations; computation performs $\rho^l \cdot H_{out}W_{out} \cdot k^2 \cdot C_{in} \cdot C_{out}$ FMAs (fused multiply-accumulate operations). Under this model the overhead ratio is:

$$\hat{t}_{\text{construct}} \,/\, \hat{t}_{\text{compute}} \ \le\ 1 \,/\, (\rho^l \cdot C_{in} \cdot C_{out}). \tag{7}$$

For a typical layer with $C_{in} = C_{out} = 64$ and $\rho^l = 5\%$, this gives $\approx 0.5\%$; at $\rho^l = 1\%$ it is $\approx 2.4\%$. Construction overhead is thus negligible, and the end-to-end latency of WISEConv is dominated by the compute stage. Section V confirms that measured overhead is consistent with this bound.

C. Worklist-driven Convolution

The compute stage fuses position-level selectivity into the dense pipeline's coordinate-driven execution: it computes Eq. (1) for exactly the $\|M\|_0$ worklist entries while preserving the per-position datapath unchanged, summarized as Algorithm 2.

Algorithm 2 Worklist-driven Convolution

Require: Worklist $\mathbf{q}$, input $\mathbf{x}$, weight $\mathbf{w}$
Ensure: Output $\mathbf{y}$ at positions in $\mathbf{q}$

1: **for** each compute tile (b_m, b_n) **in parallel do**
2: $\quad\mathbf{q}' \leftarrow \mathbf{q}[\,(b_m{-}1)B_M : b_m B_M\,]$
3: $\quad\mathbf{y}' \leftarrow \mathbf{0}_{B_M \times B_N}$
4: $\quad$**for** $i = 1, \ldots, k^2$ **do**
5: $\quad\quad$**for** $j = 1, \ldots, \lceil C_{in}/B_K \rceil$ **do**
6: $\quad\quad\quad\mathbf{x}' \leftarrow \mathbf{x}\big[\,\mathcal{N}_i(\mathbf{q}'),\ (j{-}1)B_K : jB_K\,\big]$ ▷ gather input slice
7: $\quad\quad\quad\mathbf{w}' \leftarrow \mathbf{w}_i\big[\,(b_n{-}1)B_N : b_n B_N,\ (j{-}1)B_K : jB_K\,\big]$ ▷ load weight slice
8: $\quad\quad\quad\mathbf{y}' \leftarrow \mathbf{y}' + \mathbf{x}' \cdot \mathbf{w}'^{\top}$
9: $\quad\quad$**end for**
10: $\quad$**end for**
11: $\quad\mathbf{y}[\mathbf{q}'] \leftarrow \mathbf{y}'$
12: **end for**

Following standard implicit GEMM, the workload is partitioned into compute tiles: the worklist positions are divided into segments of size B_M indexed by $b_m = 1, \ldots, \lceil \|M\|_0 / B_M \rceil$, and the C_{out} output channels into segments of size B_N indexed by $b_n = 1, \ldots, \lceil C_{out}/B_N \rceil$; all compute tiles execute in parallel (line 1 in Algorithm 2). Each compute tile first loads its B_M worklist entries into $\mathbf{q}'$ (line 2). After initializing the accumulator (line 3), the compute tile iterates over the k^2 receptive-field offsets (line 4) and within each offset over C_{in} in steps of B_K (line 5): at each step, the kernel gathers the input slice $\mathbf{x}'$ at offset i from positions $\mathcal{N}_i(\mathbf{q}')$, loads the corresponding weight slice $\mathbf{w}'$ from $\mathbf{w}_i$, and accumulates the partial product (lines 6–8). Finally, the compute tile writes the accumulated result back to $\mathbf{y}$ at the worklist addresses (line 11).

The worklist order also determines access locality in the compute stage. Each compute tile processes B_M consecutive worklist entries. Because construction compacts active positions per tile, consecutive entries within each $\mathbf{q}_{P_n}$ are row-major neighbors in G_{out} whose receptive fields overlap in contiguous memory. Locality can degrade when a compute tile's B_M entries span multiple construction-tile boundaries; however, each construction tile contributes on average $\rho^l \cdot t_h t_w$ active positions, so at moderate active ratios most compute tiles fall entirely within a single segment. The resulting tile-local locality avoids the cost of a global sort while retaining sufficient regularity for efficient memory access.

IV. Implementation

WISEConv is implemented as a CUDA C++ library exposed to PyTorch via Pybind11. The core masked convolution operator serves as a drop-in replacement for `nn.Conv2d` with identical weights and hyperparameters. To support end-to-end sparse execution of complete CNN architectures, the library also provides mask-aware implementations of `ConvTranspose2d`, pooling, element-wise addition, etc. All operators accept and propagate the active mask; no model architecture modification is required and existing pre-trained weights load directly. All tensors use the channels-last (NHWC) memory layout, so that reading a spatially sparse set of positions still produces contiguous memory accesses along the channel dimension, preserving throughput.

The construction kernel maps one thread per output position; each thread tests the receptive field against M_{in} with early exit. Within each warp, `__ballot_sync` produces a bitmask of active lanes and `__popc` on masked prefixes yields each active lane's rank, giving the support set $\mathcal{I}_n$ (the ordered indices of active positions) without a sort or prefix-sum pass. One warp-elected thread performs a single `atomicAdd` to reserve a contiguous worklist segment; each active lane writes its index at the reserved base plus its rank.

The compute kernel follows a Cutlass-style [21] implicit-GEMM structure with double-buffered shared memory and asynchronous global-to-shared loads (`cp.async` on Ampere+). Tile configurations (B_M, B_N, B_K) are selected via Triton-style [22] offline autotuning: for each unique layer shape (spatial dimensions, channels, kernel size), a one-time profiling sweep determines the best-performing tile, which is then reused across all inference calls. The WISEConv output is bit-identical to dense cuDNN convolution for all active positions.

TABLE I
Evaluation workloads.

	UNet	**YOLO**	**ResNet**
Task	Optical flow	Object detection	Depth estimation
Mask source	EvConv	DeltaCNN	DGNet
Dataset	MVSEC	MOT16	KITTI
Resolution	346×260	640×640	1280×384
#Parameters	TODO	TODO	TODO
Dataset size	TODO	TODO	TODO

V. Evaluation

A. Experimental Setup

We evaluate WISEConv on four GPU platforms, three robotic perception tasks with distinct mask sources, and three baselines representing the dense, tile-skipping, and gather-scatter paradigms.

1) Platforms: Experiments run on two embedded platforms representative of onboard robotic deployment and two desktop-class GPUs that test generality: Jetson Xavier NX (8 GB, CUDA 11.4), Jetson AGX Orin (64 GB, CUDA 11.4), RTX 4070 Laptop (CUDA 12.1), and RTX 3080 Desktop (CUDA 12.1).

2) Models, Tasks, and Mask Sources: Table I summarizes the three evaluation workloads. Each pairs a robotic perception task with a mask source drawn from a different family (Section II-B1).

Optical flow. We use EVFlowNet [23], a UNet encoder-decoder, with EvConv [4] as the mask source. We follow the standard EvConv evaluation protocol on the MVSEC dataset [24] (346×260 resolution).

Object detection. We use YOLOv11 [25] with temporal frame differences as the mask source (DeltaCNN [6]): consecutive frames are differenced and positions exceeding a threshold are marked active. We evaluate on the MOT16 dataset [26] with 640×640 input resolution.

Depth estimation. We use a ResNet-34 encoder-decoder [27] with DGNet [7] learned spatial gating as the mask source: a lightweight gating module predicts task-relevant regions per frame. We evaluate on KITTI [28] (1280×384 resolution).

EVFlowNet [23] and YOLOv11 use their respective official pretrained weights; EvConv [4] and DeltaCNN [6] are inference-time mask sources that do not modify model weights. For DGNet [7], the gating module is jointly trained with the backbone following the default DGNet configuration.

3) Baselines: We compare against three baselines, all running in FP32 on identical hardware and inputs:

- **Dense**: cuDNN [17] implicit GEMM with best-algorithm selection via `cudnnFind`.
- **Tile skipping**: DeltaCNN [6] execution engine. DeltaCNN includes a delta-propagation optimization specific to temporal frame differences (computing and propagating only output deltas). We enable delta propagation for the temporal frame difference task and disable it for the event-camera and learned-gating tasks since they are not applicable.
- **Gather-scatter**: MMCV [29] `MaskedConv2d` operator, which performs `masked_im2col` (gather active patches), matrix multiply, and `masked_col2im` (scatter back).

4) Metrics and Methodology: We report four metrics: (1) end-to-end latency (ms), (2) useful-compute ratio η (Section II-C), (3) throughput (GFLOPS, giga floating-point operations per second), and (4) task accuracy: Average Endpoint Error (AEE, pixels) for optical flow, mean Average Precision (mAP, %) for object detection, and Root Mean Square Error (RMSE, meters) for depth estimation. All latency measurements use CUDA events with device synchronization, averaged over 3 repetitions after 3 warm-up runs. We report standard deviation.

B. End-to-End Latency

C. Efficiency Analysis

D. Construction Overhead

E. Task Accuracy

ACKNOWLEDGMENTS

REFERENCES

[1] D. Wofk, F. Ma, T. Yang, S. Karaman, and V. Sze, "Fastdepth: Fast monocular depth estimation on embedded systems," in *Proc. IEEE Int. Conf. Robot. Autom. (ICRA)*, 2019, pp. 6101–6108.

[2] R. J. Bouwmeester, F. Paredes-Vallés, and G. C. H. E. de Croon, "Nanoflownet: Real-time dense optical flow on a nano quadcopter," in *Proc. IEEE Int. Conf. Robot. Autom. (ICRA)*, 2023, pp. 1996–2003.

[3] M. Sundermeyer, A. Mousavian, R. Triebel, and D. Fox, "Contact-graspnet: Efficient 6-dof grasp generation in cluttered scenes," in *Proc. IEEE Int. Conf. Robot. Autom. (ICRA)*, 2021, pp. 13 438–13 444.

[4] S. Durvasula, Y. Guan, and N. Vijaykumar, "Ev-conv: Fast CNN inference on event camera inputs for high-speed robot perception," *IEEE Robotics Autom. Lett.*, vol. 8, no. 6, pp. 3174–3181, 2023.

[5] N. Messikommer, D. Gehrig, A. Loquercio, and D. Scaramuzza, "Event-based asynchronous sparse convolutional networks," in *Proc. Eur. Conf. Comput. Vis. (ECCV)*, 2020, pp. 415–431.

[6] M. Parger, C. Tang, C. D. Twigg, C. Keskin, R. Wang, and M. Steinberger, "Deltacnn: End-to-end CNN inference of sparse frame differences in videos," in *Proc. IEEE/CVF Conf. Comput. Vis. Pattern Recognit. (CVPR)*, 2022, pp. 12 487–12 496.

[7] F. Li, G. Li, X. He, and J. Cheng, "Dynamic dual gating neural networks," in *Proc. IEEE/CVF Int. Conf. Comput. Vis. (ICCV)*, 2021, pp. 5310–5319.

[8] M. Ren, A. Pokrovsky, B. Yang, and R. Urtasun, "Sbnet: Sparse blocks network for fast inference," in *Proc. IEEE Conf. Comput. Vis. Pattern Recognit. (CVPR)*, 2018, pp. 8711–8720.

[9] A. Habibian, D. Abati, T. S. Cohen, and B. E. Bejnordi, "Skip-convolutions for efficient video processing," in *Proc. IEEE Conf. Comput. Vis. Pattern Recognit. (CVPR)*, 2021, pp. 2695–2704.

[10] Z. Xie, Z. Zhang, X. Zhu, G. Huang, and S. Lin, "Spatially adaptive inference with stochastic feature sampling and interpolation," in *Proc. Eur. Conf. Comput. Vis. (ECCV)*, 2020, pp. 531–548.

[11] L. Cavigelli, P. Degen, and L. Benini, "Cbinfer: Change-based inference for convolutional neural networks on video data," in *Proc. Int. Conf. Distrib. Smart Cameras (ICDSC)*, 2017, pp. 1–8.

[12] Z. Teed and J. Deng, "RAFT: recurrent all-pairs field transforms for optical flow," in *Proc. Eur. Conf. Comput. Vis. (ECCV)*, 2020, pp. 402–419.

[13] C. Wang, A. Bochkovskiy, and H. M. Liao, "Yolov7: Trainable bag-of-freebies sets new state-of-the-art for real-time object detectors," in *Proc. IEEE/CVF Conf. Comput. Vis. Pattern Recognit. (CVPR)*, 2023, pp. 7464–7475.

[14] C. Yu, J. Wang, C. Peng, C. Gao, G. Yu, and N. Sang, "Bisenet: Bilateral segmentation network for real-time semantic segmentation," in *Proc. Eur. Conf. Comput. Vis. (ECCV)*, 2018, pp. 334–349.

[15] C. Wang, D. Xu, Y. Zhu, R. M. Martin, C. Lu, L. Fei-Fei, and S. Savarese, "Densefusion: 6d object pose estimation by iterative dense fusion," in *Proc. IEEE Conf. Comput. Vis. Pattern Recognit. (CVPR)*, 2019, pp. 3343–3352.

[16] A. Dosovitskiy, L. Beyer, A. Kolesnikov, D. Weissenborn, X. Zhai, T. Unterthiner, M. Dehghani, M. Minderer, G. Heigold, S. Gelly, J. Uszkoreit, and N. Houlsby, "An image is worth 16x16 words: Transformers for image recognition at scale," in *Proc. Int. Conf. Learn. Represent. (ICLR)*, 2021.

[17] S. Chetlur, C. Woolley, P. Vandermersch, J. Cohen, J. Tran, B. Catanzaro, and E. Shelhamer, "cudnn: Efficient primitives for deep learning," *CoRR*, vol. abs/1410.0759, 2014.

[18] G. Gallego, T. Delbrück, G. Orchard, C. Bartolozzi, B. Taba, A. Censi, S. Leutenegger, A. J. Davison, J. Conradt, K. Daniilidis, and D. Scaramuzza, "Event-based vision: A survey," *IEEE Trans. Pattern Anal. Mach. Intell.*, vol. 44, no. 1, pp. 154–180, 2022.

[19] Y. Yan, Y. Mao, and B. Li, "SECOND: sparsely embedded convolutional detection," *Sensors*, vol. 18, no. 10, p. 3337, 2018.

[20] H. Tang, S. Yang, Z. Liu, K. Hong, Z. Yu, X. Li, G. Dai, Y. Wang, and S. Han, "Torchsparse++: Efficient training and inference framework for sparse convolution on gpus," in *Proc. IEEE/ACM Int. Symp. Microarchitecture (MICRO)*, 2023, pp. 225–239.

[21] "CUTLASS: CUDA templates and python dsls for high-performance linear algebra," 2026. [Online]. Available: https://github.com/NVIDIA/cutlass

[22] P. Tillet, H. Kung, and D. D. Cox, "Triton: an intermediate language and compiler for tiled neural network computations," in *Proc. ACM Workshop Mach. Learn. Program. Lang. (MAPL@PLDI)*, 2019, pp. 10–19.

[23] A. Z. Zhu, L. Yuan, K. Chaney, and K. Daniilidis, "Ev-flownet: Self-supervised optical flow estimation for event-based cameras," in *Proc. Robot.: Sci. Syst. (RSS)*, 2018.

[24] A. Z. Zhu, D. Thakur, T. Özaslan, B. Pfrommer, V. Kumar, and K. Daniilidis, "The multi vehicle stereo event camera dataset: An event camera dataset for 3d perception," *CoRR*, vol. abs/1801.10202, 2018.

[25] "Ultralytics YOLO11," 2024, version 8.3.0. [Online]. Available: https://github.com/ultralytics/ultralytics

[26] A. Milan, L. Leal-Taixé, I. D. Reid, S. Roth, and K. Schindler, "MOT16: A benchmark for multi-object tracking," *CoRR*, vol. abs/1603.00831, 2016.

[27] C. Godard, O. M. Aodha, M. Firman, and G. J. Brostow, "Digging into self-supervised monocular depth estimation," in *Proc. IEEE/CVF Int. Conf. Comput. Vis. (ICCV)*, 2019, pp. 3827–3837.

[28] A. Geiger, P. Lenz, and R. Urtasun, "Are we ready for autonomous driving? the KITTI vision benchmark suite," in *Proc. IEEE Conf. Comput. Vis. Pattern Recognit. (CVPR)*, 2012, pp. 3354–3361.

[29] K. Chen, J. Wang, J. Pang, Y. Cao, Y. Xiong, X. Li, S. Sun, W. Feng, Z. Liu, J. Xu, Z. Zhang, D. Cheng, C. Zhu, T. Cheng, Q. Zhao, B. Li, X. Lu, R. Zhu, Y. Wu, J. Dai, J. Wang, J. Shi, W. Ouyang, C. C. Loy, and D. Lin, "Mmdetection: Open mmlab detection toolbox and benchmark," *CoRR*, vol. abs/1906.07155, 2019.